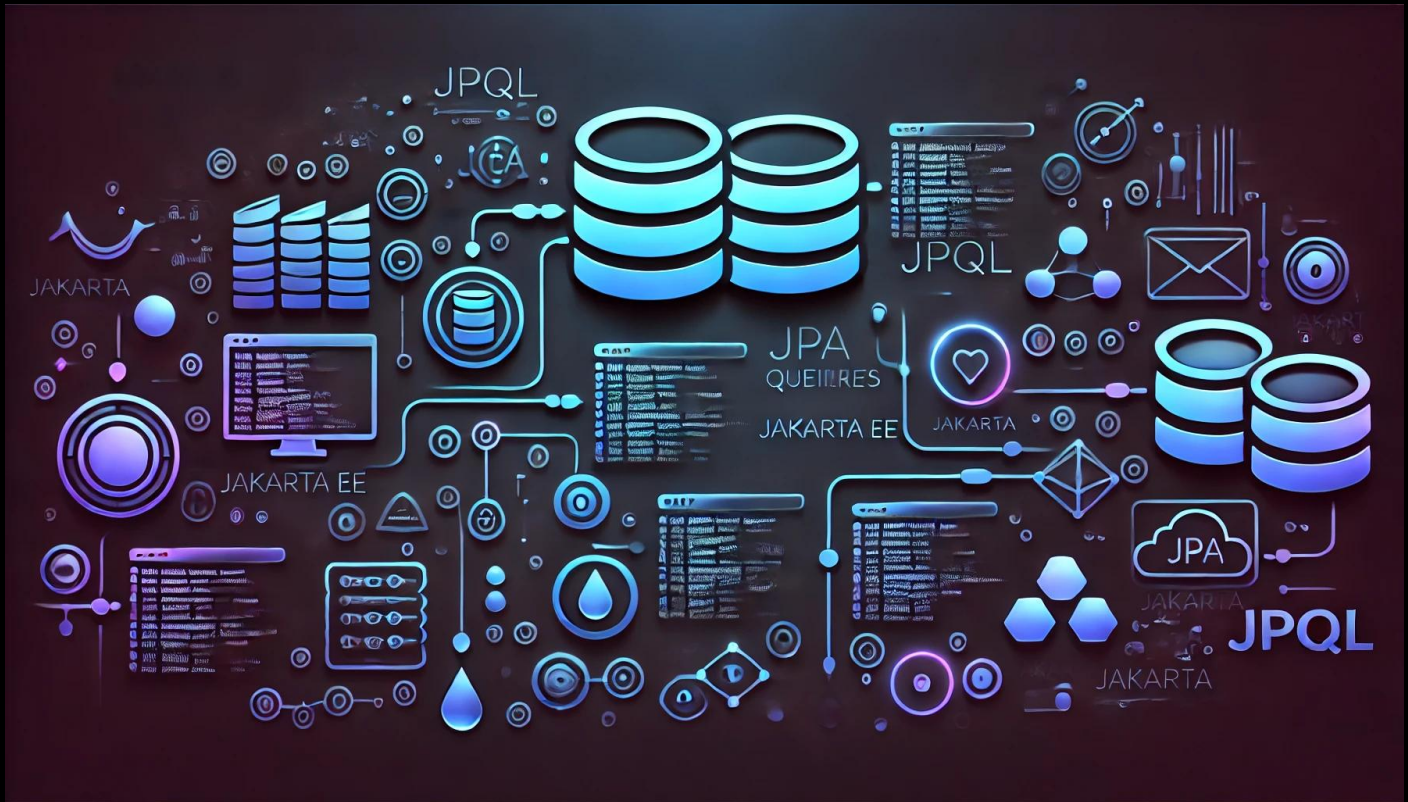


JPQL con JPA y Jakarta EE – parte 8



■ Guía: Uso de BETWEEN y ORDER BY en JPQL con JPA

1 Introducción

En esta guía aprenderemos a utilizar **BETWEEN** y **ORDER BY** en **JPQL con JPA** para filtrar y ordenar datos de manera eficiente.

✦ Objetivos:

1. **Filtrar registros** utilizando **BETWEEN** para seleccionar un rango de valores.
2. **Ordenar registros** utilizando **ORDER BY** en combinación con criterios específicos.

◆ Casos de uso:

- ✓ Seleccionar registros dentro de un rango de IDs.
- ✓ Ordenar los resultados por nombre y apellido en orden descendente.

2 Conceptos Básicos

✖ Uso de BETWEEN en JPQL

El operador `BETWEEN` se usa para seleccionar valores dentro de un rango.

Ejemplo en JPQL:

```
SELECT p FROM Persona p WHERE p.idPersona BETWEEN 1 AND 10
```

- ◆ Recupera todas las personas cuyo `idPersona` esté entre 1 y 10 (incluidos ambos extremos).
-

✖ Uso de ORDER BY en JPQL

El operador `ORDER BY` permite ordenar los resultados por una o más columnas.

Ejemplo en JPQL:

```
SELECT p FROM Persona p ORDER BY p.nombre DESC, p.apellido DESC
```

- ◆ Ordena primero por `nombre` en **orden descendente** (`DESC`).
 - ◆ Luego, si hay nombres iguales, ordena por `apellido` en **orden descendente**.
-

3 Consulta: Filtrar registros con BETWEEN

✖ Queremos obtener todas las personas cuyo `idPersona` esté entre 1 y 10.

◆ Código actualizado

```
//12. Uso de between
```

```
log.info("\n12. Filtrar registros con BETWEEN");
jpql = "SELECT p FROM Persona p WHERE p.idPersona BETWEEN 1 AND 10";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);
```

🗉 Explicación:

- `BETWEEN 1 AND 10` filtra las personas con ID dentro del rango de 1 a 10.

- Se obtiene una **lista de personas** con `getResultList()`.
-

4 Consulta: Ordenar registros con ORDER BY

✦ Queremos obtener todas las personas con `idPersona > 1`, ordenadas por nombre y apellido en orden descendente.

◆ Código actualizado

```
//13. Uso del ordenamiento
log.info("\n13. Ordenar registros con ORDER BY");
jpql = "SELECT p FROM Persona p WHERE p.idPersona > 1 ORDER BY p.nombre DESC, p.apellido DESC";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);
```

🗨 Explicación:

- `idPersona > 1` filtra personas con ID mayor a 1.
 - `ORDER BY p.nombre DESC, p.apellido DESC`
 - ◆ Ordena primero por nombre en **orden descendente**.
 - ◆ Luego, ordena por apellido en **orden descendente** si los nombres son iguales.
-

5 Código Completo Actualizado

```
package sga.cliente.jpql;

import java.util.List;
import jakarta.persistence.*;
import java.util.Iterator;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class PruebaJPQL {

    private static final Logger log = LoggerFactory.getLogger(PruebaJPQL.class);

    public static void main(String[] args) {
        String jpql = null;
    }
}
```

```
List<Persona> personas = null;

EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
try (EntityManager em = emf.createEntityManager()) {

    //1. Consulta de todos los objetos de tipo Persona
    log.info("\n1. Consulta de todas las Personas");
    jpql = "select p from Persona p";
    personas = em.createQuery(jpql, Persona.class).getResultList();
    mostrarPersonas(personas);

    //2. Consulta de una persona por ID
    log.info("\n2. Consulta de Persona con ID = 1");
    jpql = "SELECT p FROM Persona p WHERE p.idPersona = 1";
    Persona persona = em.createQuery(jpql, Persona.class).getSingleResult();
    log.info("Persona encontrada: " + persona);

    //3. Consulta de una persona por nombre
    log.info("\n3. Consulta de Persona por nombre 'Ivonne'");
    jpql = "SELECT p FROM Persona p WHERE p.nombre = 'Ivonne'";
    personas = em.createQuery(jpql, Persona.class).getResultList();
    mostrarPersonas(personas);

    //4. Consulta de datos individuales (tupla)
    log.info("\n4. Consulta de datos individuales: nombre, apellido, email");

    jpql = "SELECT p.nombre, p.apellido, p.email FROM Persona p";
    Iterator iter = em.createQuery(jpql).getResultList().iterator();

    while (iter.hasNext()) {
        Object[] tupla = (Object[]) iter.next();
        String nombre = (String) tupla[0];
        String apellido = (String) tupla[1];
        String email = (String) tupla[2];

        log.info("Nombre: " + nombre + ", Apellido: " + apellido + ", Email: " + email);
    }

    //5. Obtiene el objeto Persona y el id, se crea un arreglo de tipo Object con 2 columnas
    log.info("\n5. Obtiene el id y el objeto Persona, se crea un arreglo de tipo Object con 2
columnas");
    Object[] tupla = null;
    jpql = "select p.idPersona, p from Persona p ";
    iter = em.createQuery(jpql).getResultList().iterator();
    while (iter.hasNext()) {
```

```
tupla = (Object[]) iter.next();
int idPersona = (int) tupla[0];
persona = (Persona) tupla[1];
log.info("ID persona:" + idPersona);
log.info("Objeto persona:" + persona);
}

//6. Consulta de todas las personas
log.info("\n6. Consulta de los ids de todas las personas");
jpql = "SELECT p.idPersona FROM Persona p";
List<Integer> ids = em.createQuery(jpql, Integer.class).getResultList();
ids.forEach(idPersona -> log.info("ID persona: " + idPersona));

//7. Regresa el valor minimo y maximo del idPersona (scaler result)
log.info("\n7. Regresa el valor minimo y maximo del idPersona (scaler result)");
jpql = "select MIN(p.idPersona) as MinId, MAX(p.idPersona) as MaxId, COUNT(p.idPersona) as
Contador from Persona p";
iter = em.createQuery(jpql).getResultList().iterator();
while (iter.hasNext()) {
    tupla = (Object[]) iter.next();
    Integer idMin = (Integer) tupla[0];
    Integer idMax = (Integer) tupla[1];
    Long count = (Long) tupla[2];
    log.info("idMin:" + idMin + ", idMax:" + idMax + ", count:" + count);
}

//8. Cuenta los nombres de las personas que son distintos
log.info("\n8. Cuenta los nombres de las personas que son distintos");
jpql = "select COUNT(distinct p.nombre) from Persona p";
Long contador = (Long) em.createQuery(jpql).getSingleResult();
log.info("no. de personas con nombre distinto:" + contador);

//9. Concatena y convierte a mayusculas el nombre y apellido
log.info("\n9. Concatena y convierte a mayusculas el nombre y apellido");
jpql = "select CONCAT(p.nombre, ' ', p.apellido) as Nombre from Persona p";
List<String> nombres = em.createQuery(jpql).getResultList();
for (String nombreCompleto : nombres) {
    log.info(nombreCompleto);
}

//10. Obtiene el objeto persona con id igual al parametro proporcionado
log.info("\n10. Obtener objeto Persona por ID");
int idPersona = 1; // Podemos cambiar este valor dinámicamente
// Definir consulta con parámetro
jpql = "SELECT p FROM Persona p WHERE p.idPersona = :id";
```

```

Query consulta = em.createQuery(jpql);
consulta.setParameter("id", idPersona);
// Ejecutar consulta y obtener resultado único
persona = (Persona) consulta.getSingleResult();
log.info("Persona encontrada: " + persona);

//11. Obtiene las personas que contengan una letra a en el nombre, sin importar si es
mayusculas o minuscula
log.info("\n11. Obtener personas con letra 'a' en el nombre (ignorar
mayusculas/minusculas)");
// Definir consulta con LIKE y UPPER()
jpql = "SELECT p FROM Persona p WHERE UPPER(p.nombre) LIKE UPPER(:parametro)";
String parametroString = "%a%"; // Busca la letra 'a' en cualquier posición
consulta = em.createQuery(jpql);
consulta.setParameter("parametro", parametroString);
personas = consulta.getResultList();
mostrarPersonas(personas);

//12. Uso de between
log.info("\n12. Filtrar registros con BETWEEN");
jpql = "SELECT p FROM Persona p WHERE p.idPersona BETWEEN 1 AND 10";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//13. Uso del ordenamiento
log.info("\n13. Ordenar registros con ORDER BY");
jpql = "SELECT p FROM Persona p WHERE p.idPersona > 1 ORDER BY p.nombre DESC, p.apellido
DESC";

personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

}
}

private static void mostrarPersonas(List<Persona> personas) {
    for (Persona p : personas) {
        log.info("Persona: " + p);
    }
}
}

```

7 Resultado Esperado

Si ejecutamos la clase PruebaJPQL, veremos en la consola:

```
12. Filtrar registros con BETWEEN
[EL Fine]: sql: ServerSession(412925308)--Connection(696165690)--SELECT id_persona, APELLIDO, EMAIL, NOMBRE,
TELEFONO FROM PERSONA WHERE (id_persona BETWEEN ? AND ?)
    bind => [1, 10]
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=1, nombre=Rafael, apellido=Juarez,
email=rjuarez@mail.com, telefono=55667788}
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=2, nombre=Ivonne, apellido=Lara,
email=ilara@mail.com, telefono=55443322}
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=3, nombre=Maria, apellido=Esparza,
email=mesparza@mail.com, telefono=55112233}
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=5, nombre=Hugo, apellido=Hernandez,
email=hhernandez@mail.com, telefono=55778822}
[main] INFO sga.cliente.jpql.PruebaJPQL -
13. Ordenar registros con ORDER BY
[EL Fine]: sql: ServerSession(412925308)--Connection(696165690)--SELECT id_persona, APELLIDO, EMAIL, NOMBRE,
TELEFONO FROM PERSONA WHERE (id_persona > ?) ORDER BY NOMBRE DESC, APELLIDO DESC
    bind => [1]
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=3, nombre=Maria, apellido=Esparza,
email=mesparza@mail.com, telefono=55112233}
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=2, nombre=Ivonne, apellido=Lara,
email=ilara@mail.com, telefono=55443322}
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=5, nombre=Hugo, apellido=Hernandez,
email=hhernandez@mail.com, telefono=55778822}
```

8 Conclusión

🔗 En esta guía aprendimos a utilizar **BETWEEN** y **ORDER BY** en JPQL con JPA para filtrar y ordenar registros de manera eficiente.

- ◆ **Puntos clave:**
- ✓ **BETWEEN** se usa para filtrar valores dentro de un rango.
- ✓ **ORDER BY** permite ordenar los resultados por múltiples columnas.
- ✓ **JPQL** es una poderosa herramienta para consultas dinámicas.

🚀 ¡Ahora puedes aplicar estas técnicas en tus proyectos!

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)